



Ficha prática #4

Processos e *threads*

1. Introdução ao tema

Neste guião vamos estudar os processos e as *threads*, as abstrações básicas de computação dos sistemas operativos modernos. O uso destas abstrações nos nossos programas permite-nos executar tarefas de forma *concorrencial* (ou mesmo *paralela*, se o sistema for multiprocessador).

1.1. Processos

Um processo é uma instância de um programa em execução. Quando um programa é executado, o *kernel*:

- a) carrega o código do programa para memória;
- b) reserva espaço em memória para as variáveis do programa; e
- c) prepara as estruturas necessárias para armazenar informação sobre o processo, designadamente o bloco de controlo do processo, que contém a sua identificação (*Process ID*, ou *PID*), o seu estado, etc.

O processo *init* é o pai de todos os processos e em *PID* igual a 1. Todos os processos derivam desse. Como se pode ver na figura 1, a memória usada por um processo é dividida em vários segmentos:

- d) **texto**: contém as instruções do programa;
- e) **dados**: contém as variáveis globais estáticas;
- f) **heap**: área onde o programa pode reservar memória (variáveis globais) de uma forma dinâmica;
- g) **stack** (ou pilha): memória usada na chamada de funções, para passar parâmetros (de entrada e de saída) e para armazenar as variáveis temporárias usadas pelas funções. A pilha cresce e encolhe à medida que as funções são chamadas.



Figura 1. Organização de um processo em memória

1.1.1 Criação de processos em Python

Para criar um processo usa-se a chamada ao sistema¹ `fork()`. Esta chamada ao sistema está acessível em Python através do módulo `os`:

```
import os
```

```
pid = os.fork()
```

Ao fazer-se esta chamada ao sistema passam a existir dois processos – o **pai** e o **filho** – e a execução do programa continua, em ambos os processos, no ponto em que o `fork()` retorna. O processo chamador é o **pai**, e o novo processo é o **filho**. Caso não haja erros, o `fork()` retorna o PID do filho ao processo pai, e zero ao processo filho. Caso haja erros, é gerada a exceção `OSError`. O processo-filho é uma cópia do processo-pai². Assim, qualquer um dos dois processos pode alterar as variáveis definidas em qualquer dos seus segmentos sem afetar o outro processo. Ou seja, o pai e o filho são **independentes**. A figura 2 ilustra esquematicamente estes conceitos.

Reparem como os segmentos de memória do filho são uma cópia exata dos segmentos do pai, mas são totalmente independentes. Para retornar o PID de um processo e do seu pai podem usar-se as seguintes funções:

```
os.getpid()
```

```
os.getppid()
```

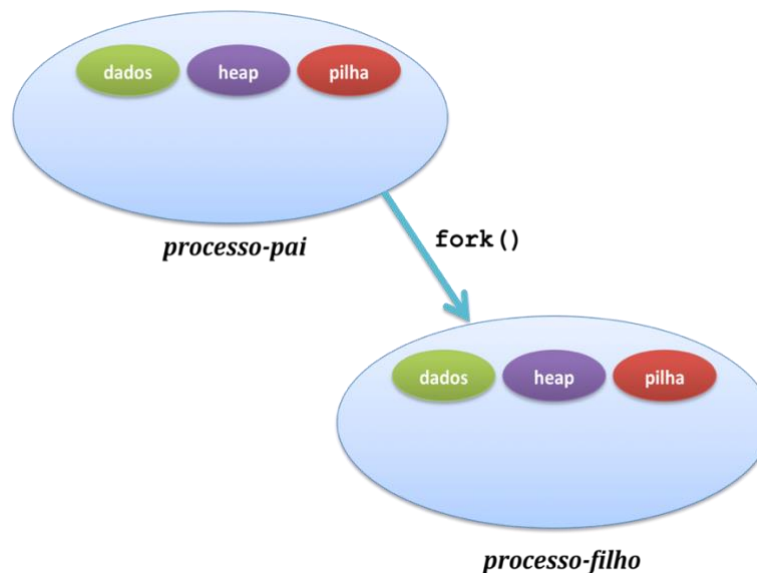


Figura 2. Chamada ao sistema `fork()`

¹ Uma chamada ao sistema (ou *system call*) serve como ponto de entrada (controlado) para o *kernel*, permitindo ao processo pedir ao *kernel* que execute alguma ação. Para o programador, no entanto, uma chamada ao sistema é exatamente igual à invocação de uma qualquer função em Python. Mas “atrás das cortinas” as coisas são um pouco diferentes...

² Esta cópia é muitas vezes concretizada de forma diferente em sistemas diferentes.



O próximo código em Python³ ilustra o uso da chamada ao sistema `fork`⁴. Para executar um programa em Python: `python3 abc.py`

```
import os, time
myVar = 10
def funcao():
    global myVar
    myVar = 1000
    time.sleep(5)

pid=os.fork()

if pid == 0: #filho
    funcao()
else: #pai
    time.sleep(5)
print("Sou o PID = " + str(os.getpid()) + ". myVar = " + str(myVar))
```

Nota: executem este programa em background e verifiquem que os processos são de facto criados com os PIDs esperados (os `sleep()` introduzidos no código permitem que tenham tempo para os visualizar). Assumindo que gravam este código num ficheiro `processos.py`, basta correr estes comandos na shell:

```
$ python processos.py &
```

```
$ ps
```

O resultado da execução deste programa é o seguinte:

```
Sou o PID = 16000; myVar = 10
```

```
Sou o PID = 16001; myVar = 1000
```

Reparem como o filho alterou a variável global `myVar` mas isso não afetou a variável com o mesmo nome do pai. Elas são globais em cada processo⁵ mas são, de facto, *independentes*. Os filhos também podem, por sua vez, criar outros filhos, como no exemplo que se segue.

³ O código em Python apresentado vai ser sempre colocado dentro de uma caixa para se distinguir dos comandos a executar no terminal (e dos *shell-scripts*).

⁴ Os exemplos apresentados incluem muitas vezes a função `sleep()` da biblioteca `time`. Esta função permite suspender um processo por um período especificado em segundos, e é por isso muito útil para dar tempo de se observarem os processos no sistema.

⁵ Quando se quer usar uma variável global numa função isso deve ser feito explicitamente, como na **linha de código a vermelho**



```
import os, time
pid_filho=os.fork()
if pid_filho == 0: #filho
    pid_neto=os.fork()
    if pid_neto == 0: #neto
        print("Eu sou o neto. PID = " + str(os.getpid()))
        time.sleep(5)
    else: #filho
        print("Eu sou o filho. PID = " + str(os.getpid()))
        time.sleep(5)
else: #pai
    print("Eu sou o pai. PID = " + str(os.getpid()))
    time.sleep(5)
```

O resultado da execução deste programa é o seguinte:

Eu sou o pai. PID = 2562

Eu sou o filho. PID = 2563

Eu sou o neto. PID = 2564

A árvore dos processos gerados por este programa e o seu fio de execução são apresentados na figura 3.

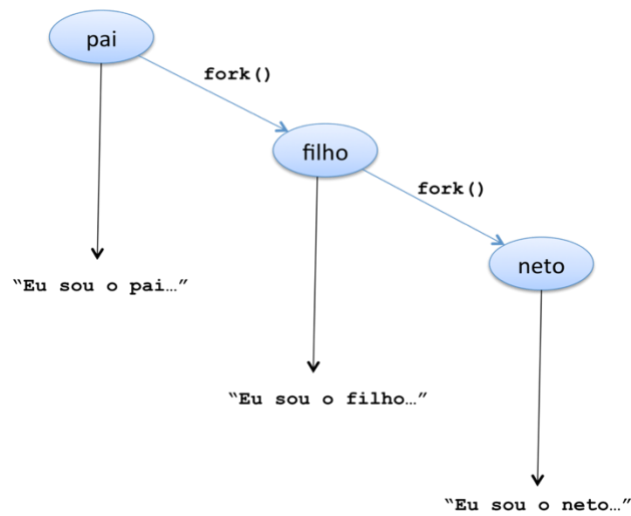


Figura 3. Árvore de processos



1.1.2 Execução de um novo programa

Como vimos nos exemplos anteriores, o processo-filho pode executar funções diferentes no mesmo código que partilha com o pai. No entanto, é muito mais comum o processo-filho executar um programa *completamente novo*. Para tal existe a família de chamadas ao sistema `exec()`. Há várias funções de biblioteca que chamam a `exec()`, tais como a `execlp()`, `execvp()`, entre outras. Estas diferem apenas pela forma como são passados os argumentos ao novo programa e pouco mais. Dada esta explicação, neste guião vamos usar apenas uma destas funções: a `execlp()`:

```
os.execlp(file, arg0, arg1, ...)
```

Nesta função, o `file` indica o ficheiro que é para ser executado. De seguida incluem-se todos os argumentos a passar ao novo programa, separados por vírgulas. Quando se corre esta função, o código do processo é substituído pelo novo programa. Há assim uma *separação radical* entre o pai e o filho.

Veja-se o exemplo que se segue, onde são criados 5 filhos que se vão separar radicalmente do pai correndo um novo programa, `filho.py`, passando-lhe o argumento `numFilho`.

```
import os, time
for i in range(5):
    numFilho = i+1

    pid=os.fork()

    if pid == 0: #filho
        os.execlp("python3", "python3", "filho.py", str(numFilho))
    else:
        print "PID do filho " + str(numFilho) + " = " + str(pid)
```

O programa `filho.py` encontra-se ilustrado a seguir:

```
import os, sys

print "Ola a partir do filho numero " + sys.argv[1] + ". PID = " + str(os.getpid())
```

Nota: `sys.argv[1]` contém o argumento passado ao programa `filho.py`. O resultado da execução deste programa é o seguinte:

PID do filho 1 = 16317

PID do filho 2 = 16318

PID do filho 3 = 16319

PID do filho 4 = 16320

PID do filho 5 = 16321

Ola a partir do filho numero 2. PID = 16318

Ola a partir do filho numero 1. PID = 16317

Ola a partir do filho numero 3. PID = 16319

Ola a partir do filho numero 4. PID = 16320

Ola a partir do filho numero 5. PID = 16321

Observe que a ordem em que os filhos executam é “aleatória”, já que depende do escalonamento do CPU, algo sobre o qual o utilizador não tem controlo.

1.2. *Threads*

Tal como os processos, as *threads* são um mecanismo que permite a uma aplicação executar várias tarefas simultaneamente (isto é, de forma *concorrente*). Quando um programa é iniciado, o processo consiste numa única *thread*, normalmente chamada de “*main thread*”, mas cada processo pode depois ter múltiplas *threads* de execução (fios de execução). A *thread* corre o mesmo código do processo e **partilha** as variáveis globais (o segmento de dados e o *heap*⁶). Mas cada *thread* mantém a sua própria pilha. Ao contrário dos processos-filho, as *threads* podem assim comunicar entre si muito facilmente utilizando as variáveis globais partilhadas. Sendo partilhadas, a desvantagem é que é necessário *sincronizar* o acesso às mesmas. A figura seguinte ilustra a organização da memória de um processo após a criação de uma *thread*. Reparem nos segmentos partilhados – dados e *heap* – e nos segmentos que são locais a cada uma das *threads* – a pilha.

⁶ Notem a diferença em relação à criação de processos-filho, onde estes dois segmentos eram *copiados*, passando a ser segmentos *independentes*.

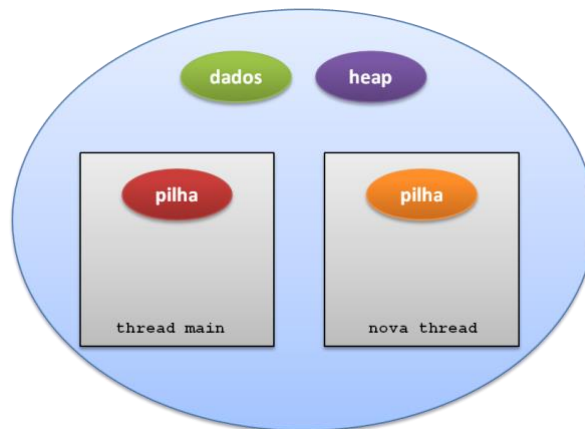


Figura 4. Criação de nova *thread*

A função construtor `Thread()` do módulo `threading` permite criar uma *thread* num processo.

```
import Thread
```

```
newThread = threading.Thread(group, target, name, args, kwargs)
```

Quando a nova *thread* começar a ser executada vai correr a função definida em *target*, com os argumentos passados através do tuplo *args*. Os outros argumentos não serão explorados nesta aula. A execução de uma *thread* começa com:

```
newThread.start()
```

Notem que após a criação de uma nova *thread* o programa não tem qualquer garantia acerca de qual a próxima *thread* que vai utilizar o CPU. Portanto, o programador não tem nenhuma garantia relativamente à ordem em que vão ser executadas as instruções do programa.

Quando a *thread* principal correr o método `join()` fica à espera que a *thread* previamente lançada termine.

```
newThread.join()
```

O próximo código em Python ilustra o uso de *threads* num programa equivalente ao primeiro programa apresentado neste documento.



```
import time
from threading import Thread

myVar = 10

def funcao():
    global myVar
    myVar = 1000
    print("Sou a nova thread. myVar= " + str(myVar))

newT = Thread(target=funcao)
newT.start()
newT.join()
print("Sou a main thread. A nova terminou. myVar = " + str(myVar))
```

Apresenta-se agora o resultado da execução deste programa:

Sou a nova thread. myVar= 1000

Sou a main thread. A nova terminou. myVar = 1000

Como se pode confirmar, ao contrário do caso em que se criam processos novos (recordar primeiro exemplo apresentado neste guião), uma nova *thread* partilha as variáveis globais com a *thread* principal (main). Por isso, quando se altera a variável global, esta alteração é visível pela *thread* main.

Terminamos com um exemplo um pouco mais complexo do uso de *threads*, onde se tenta mostrar a concorrência entre *threads*.

```
import time
from threading import Thread

def somatorio(i):
    soma=0
    for num in range(1, i+1):
        soma += num
```




```
time.sleep(1) #simula computacao complexa
print"soma="+str(soma)+" || "

def fact(i):
    prod=1
    for num in range(1, i+1):
        prod *= num
    time.sleep(1) #simula computacao complexa
    print "factorial = " + str(prod) + " || "

t = []

start_time = time.time()

t.append(Thread(target=somatorio, args=(15,)))
t.append(Thread(target=fact, args=(15,)))

t[0].start()
t[1].start()

for i in range(2):
    t[i].join()

elapsed_time = time.time() - start_time

print "Threads terminaram em " + str(elapsed_time) + " segundos"
```

O resultado da execução deste programa é o seguinte:

soma = 120 ||

factorial = 1307674368000 ||

Threads terminaram em 1.00152802467 segundos

O código inclui *duas* chamadas à função `time.sleep(1)`. Seria, assim, de esperar que as threads terminassem em apenas 1 segundo, como aconteceu?

1.3. Módulo multiprocessing

Para evitar incoerências em algumas estruturas de dados internas do interpretador Python, apenas uma *thread* pode correr no interpretador de cada vez. Há um Global Interpreter Lock (GIL) que controla a execução das *threads* para garantir que isto acontece. Em consequência, devido à existência do GIL, múltiplas *threads* criadas em Python não podem correr em paralelo em sistemas multiprocessador.

Para quem quer ultrapassar este problema e paralelizar o seu programa, mas ainda assim usar a API simples das *threads* apresentada na secção anterior, pode usar o módulo `multiprocessing`. A API é praticamente igual à do módulo `multithreading`, mas, com este modo, em vez de lançarmos novas threads lançamos novos processos, permitindo que o programa corra, de facto, em paralelo num sistema multiprocessador. O exemplo com que terminamos este documento ilustra o primeiro exemplo de threads apresentado na secção anterior, mas correndo novos processos. Como se pode verificar, as alterações no código são mínimas (a **vermelho**).

```
import time
from multiprocessing import Process

myVar = 10

def funcao():
    global myVar
    myVar = 1000
    print "Sou a nova thread. myVar= " + str(myVar)

newT = Process(target=funcao)
newT.start()
newT.join()

print "Sou a main thread. A nova terminou. myVar = " + str(myVar)
```

Apresenta-se agora o resultado da execução deste programa:

Sou a nova thread. myVar= 1000



Sou a main thread. A nova terminou. myVar = 10

Como esperado, as variáveis globais de cada processo são agora completamente independentes.

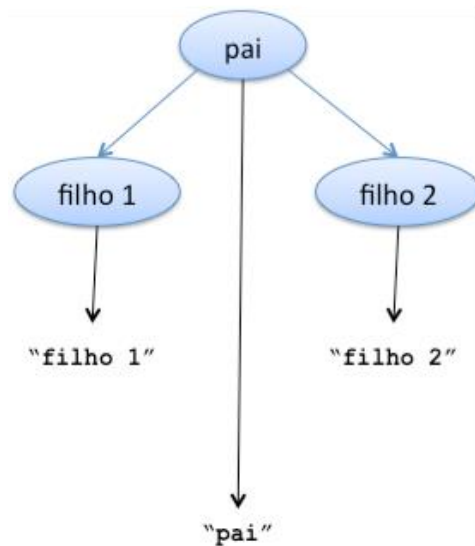
2. Exercícios fundamentais

Descarregue da página da disciplina o ficheiro tp04.zip. Descompacte os ficheiros – triplo_fork.py, fork_varglobal.py, proc_thread.py e num_fichdir.sh – para uma pasta à sua escolha.

1. Considere a execução do programa presente no ficheiro fonte triplo_fork.py.

- Quantos processos são criados no total? Justifique.
- Quantos processos são criados especificamente pelo programa? Justifique.
- Desenhe no caderno a árvore de processos deste programa.

2. Escreva um programa em Python dois_filhos.py que crie os processos necessários e execute os comandos print de forma a gerar a seguinte árvore de processos.



3. Considere a execução do programa presente no ficheiro fonte fork_varglobal.py.

- Interprete o resultado da execução deste programa.
- Acrescente duas linhas de código a este programa antes da instrução n++; de forma que a saída no terminal passe a ser:

```
hello: 1  
hello: 2
```



4. Considere a execução do programa presente no ficheiro fonte `proc_thread.py`

- a) Comente o resultado da execução deste programa.
- b) Esta versão do programa cria um novo processo para correr a função `calc_sum`. Altere este programa de forma que, em vez de criar um novo processo, o programa crie uma *thread* para executar a mesma função. Comente o resultado da execução deste novo programa.
- c) Altere o programa desenvolvido em b) de forma a, novamente, criar um processo para correr a função `calc_sum`. Desta vez, use a API do módulo `multiprocessing` para o efeito. Comente também o resultado da execução deste programa.

5. O *shell script* `num_fichdir.sh` permite contar o número total de ficheiros e diretórios da pasta corrente. Crie um novo programa em Python chama `_prog.py` que crie um novo processo para correr este script.

3. Exercícios complementares

Os alunos que terminem com sucesso os exercícios fundamentais são convidados agora a resolver alguns exercícios relacionados com a criação e gestão de processos. O ficheiro `ec04.tar.gz`, também disponível na página da disciplina, contém os ficheiros fonte aqui referidos.

- 1. Considere o programa no ficheiro `fork1.py`. Corra-o. Interprete o resultado. Porque razão são escritas 7 linhas no monitor quando só há 3 instruções `print`?**
- 2. Considere o código presente no ficheiro `fork2.py`. Corra-o.**
 - a) Indique quantos processos são criados durante a execução deste programa. Justifique.
 - b) Indique e interprete os resultados que são escritos para *stdout*.
 - c) Altere o programa de modo a que o processo pai escreva para *stdout* o dobro da variável *var* e que o processo filho escreva metade.